

FOR SOFTWARE ENGINEERS: PLAN IT AT THE START, NOT WHEN YOU NEED IT

Handoffs

What survives a session, a model, and a Monday

By the time you need a handoff, the thing that would have made it possible is the exact thing you have just lost.

The model you depend on is unavailable. Not gone forever: an outage, a provider problem, or the far more likely one, a usage limit at four in the afternoon. Does your work stop?

Three different problems get called the same word, exactly two things cross a session boundary on their own, and there is a thirty second test that tells you whether yours would survive.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Three kinds of handoff, and why they fail for one shared reason
- Exactly what crosses a session boundary without you doing anything
- The four primitives, including the one almost nobody knows
- What belongs in CLAUDE.md, by a sharper test than "is this useful"
- Git history as the handoff that outlives the tool entirely

Out of scope

- Context management inside one session, which is its own episode
- Any claim about a specific provider outage or its frequency
- Vendor comparison. The point is not being locked into a chat log
- Backup strategy for your repository, which is a different subject

Read the two-things-cross table first. Everything else on this page is a consequence of how short that list is.

01 What crosses a session boundary on its own

CLAUDE.md	Read at the start of every session
Auto memory	The first 200 lines or 25 KB, whichever comes first
The conversation	Does NOT cross. Each session starts fresh
Your reasoning	Does NOT cross, unless you wrote it down
Skills and MCP servers	Do NOT cross to another tool or vendor
Git history	Crosses everything, including the tool going away

That is the whole list. Every decision you argued through, every constraint you agreed, every dead end you ruled out: unless it landed in a file, it is gone.

03 Three different problems, one word

SESSION TO SESSION: today is gone and tomorrow has to pick it up. That one is about what you wrote down.

MODEL TO MODEL: a different model, or a different vendor, takes over the same job. That one is about whether your instructions are portable.

PERSON TO PERSON: a colleague inherits a branch an agent helped write, and has to understand decisions they never saw being made. That one is about whether your git history tells a story.

They feel similar and they are not, but all three fail for the same underlying reason, which is the next section.

Each session starts with a fresh context window and none of the conversation history from previous sessions.

Claude Code docs, Manage sessions, checked September 2026

04 The four primitives, and the one almost nobody knows

`/export` writes the current conversation out as plain text, and takes a filename. That is your transcript: portable, readable by any tool and any human.

`/memory` edits your CLAUDE.md files and shows what auto memory has actually saved. Worth looking at once, because it is rarely what you assumed.

AND THE ONE PEOPLE MISS: `/clear` takes a NAME. ``/clear refund-rounding-investigation`` labels the conversation you are walking away from, as you leave it. That label is what you see in `/resume` later.

It pairs with `/rename` rather than replacing it: `/rename` names the session you are sitting in, `/clear <name>` names the one you are walking away from. Two ends of the same session.

`/clear` accepts an optional name, which labels the conversation being cleared so it can be identified later.

Claude Code docs, Manage sessions, checked September 2026

`/resume` presents previous conversations to pick from, where a label given at clear time appears.

Claude Code docs, Manage sessions, checked September 2026

`/rename` names the session you are currently in, and `/clear <name>` names the one you are leaving. The two cover opposite ends of the same session.

Claude Code docs, Manage sessions, checked September 2026

05 The technique you have already seen without it being called a handoff

The documented pattern is: have it interview you, write the spec to a FILE, then start a FRESH SESSION to execute it.

Read that again with handoffs in your head. That IS a handoff. You are passing work from one session to another, deliberately, and it works because the thing being handed over is a written artefact rather than a conversation.

A spec travels. A conversation does not. Which makes the highest value habit here not a command at all: when a decision gets made, put it somewhere a file can hold it.

Writing a specification to a file and executing it in a fresh session is a documented working pattern.

Claude Code docs, Best practices, checked September 2026

06 What belongs in CLAUDE.md, by a sharper test

Not "is this useful". The test is: if the next session, the next model, or the next person cannot function without this, it belongs in a file.

A constraint you would be annoyed to have to re-explain. A decision that took an argument to reach. The reason a thing is done the strange way it is done.

And the counterweight is real: a bloated file means the important rules get lost. For each line, ask whether removing it would cause a mistake. Durable, not merely true.

CLAUDE.md is read as project memory at the start of every session.

Claude Code docs, Memory and CLAUDE.md, checked September 2026

07 The artefact everybody forgets, because it is not part of the tool

Your git history. It is the one thing every model, every vendor and every human can read with none of your configuration.

Commit early, and describe what you were DOING rather than what changed, because the diff already says what changed. A message that says why is a handoff note that survives the tool disappearing entirely.

This is what pays off in the third kind of handoff. The colleague inheriting the branch cannot read your conversation and was never in your session. The commit log is what they have.

Version control is outside the tool, so it is readable without any of its configuration, by anyone.

First principles, not vendor documentation



08 The honest shape of unavailability

People picture an outage. The rare case IS an outage. The common case is a usage limit, hit mid task, in the afternoon, by you.

Same problem in ordinary clothes, and the cost is identical: work stops unless the state is somewhere else.

Two things change your exposure. How your organisation routes requests, because a gateway or a cloud provider fails differently from a direct plan and several features are simply unavailable on some of them. And whether the task you are halfway through could be described to somebody else in a paragraph.

Feature availability differs across direct plans, gateways and cloud provider routes.

Claude Code docs, LLM gateways, checked September 2026

09 The split, and it is about timing rather than knowledge

A junior handles a handoff when a handoff is needed, which is the one moment it cannot be done well, because the context has already gone.

A senior sets it up at the START, when it costs almost nothing. Label the conversation when you clear it. Write the spec to a file. Put the constraint in CLAUDE.md. Commit with a message that says why.

None of that is extra work. It is the same work, written down instead of remembered.

The preparation that makes a handoff possible has to exist before the handoff is needed, because it is made of the context the handoff has lost.

First principles, not vendor documentation

The thirty second test, on your own project, right now

1. If this tool disappeared for a day, what would I rebuild out of my own head?
2. Not what would be inconvenient. What would I genuinely reconstruct from memory?
3. Is that list written down anywhere, or is it in a window?
4. Which constraint would I be annoyed to re-explain tomorrow?
5. Which decision took an argument, and where is that argument recorded?
6. Would removing this CLAUDE.md line cause a mistake, or is it just true?
7. Do my commit messages say why, or only what?
8. Could a colleague inheriting this branch follow it without me?
9. Do I label conversations when I clear them?
10. Could I describe my current half-finished task in one paragraph?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

Manage sessions

<https://code.claude.com/docs/en/manage-sessions>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.